

Highlights

Efficient Communication Between a Non-Transformer Perception Agent and a Frozen Language Model: Filtered Text Versus a Latent Channel

Alif Tasbir, Aboozar Taherkhani

- Latent channel from a non-transformer sender compared with filtered text
- Adapter carries up to 50 events at four tokens each into a frozen model
- Filtered text is more accurate; the adapter wins under batched serving
- Per-slot, per-field recoverability of the virtual tokens is measured
- A selection rule and the pre-registration log are released

Efficient Communication Between a Non-Transformer Perception Agent and a Frozen Language Model: Filtered Text Versus a Latent Channel

Alif Tasbir^a, Aboozar Taherkhani^a

^a*De Montfort University, Leicester, United Kingdom*

Abstract

The cost of messages between language-model agents is now a design constraint, especially on edge hardware. Three levers exist: say less by filtering the text, reuse computation by caching, or send a latent message instead of text. Latent communication has been studied only between transformer models, yet the agents that sense the world are often convolutional or recurrent networks. We ask which lever suits such a sender. A heartbeat classifier passes each event to a frozen Gemma 4 E4B model either as text or through a learned linear adapter that emits four virtual tokens per event. The text arms include the strongest engineering alternative, a message listing only the abnormal events. On one task, reporting the most urgent triage tier among 1 to 50 consecutive heartbeats, we compare cost, accuracy and recoverability under a pre-registered protocol with three training seeds, held-out patients and patient-level confidence intervals. Against compact text, the adapter cut prompt-processing time by 24%, 50% and 78% and time to decision by 4%, 14% and 36% at 10, 20 and 50 events, and was more accurate than calibrated text at 10 and 20 events (balanced accuracy 0.83 against 0.73). Filtered text was 0.11 to 0.17 more accurate and equally fast for one request, but its length

varies with content, so under batched serving the adapter delivered 10 to 30% more decisions per second at 12 to 28% less energy. Heart rate, interval and abnormal-run information were recoverable from the virtual tokens, and an ablation showed that passing them drove the adapter’s accuracy gain and its lower false-alarm rate. Four tokens per event was the smallest reliable setting: with fewer, the information stayed recoverable but its use became unreliable. [TODO: External test; second sender.] A sender-side filter is the better interface when the receiver’s question is known in advance; a latent channel earns its place when many events must be carried at a predictable cost.

Keywords: agent communication, latent communication, communication efficiency, heterogeneous agents, edge inference, electrocardiogram

1. Introduction

Systems of cooperating agents built on language models now spend much of their budget on talking to each other. The tokens exchanged between agents grow with every hand-off, and much of that traffic is redundant [1]. The cost shows up as latency and energy, and on edge hardware, where memory is fixed and the longest prompt sets the batch size, as a limit on how many requests can be served at all [2, 3]. Communication efficiency has therefore become a design problem in its own right [4].

The literature offers three levers. The first is to *say less*: prune redundant messages from the agent graph [1], send only the semantic packets whose expected contribution exceeds their cost [5], or compress the prompt [6, 7, 8, 9]. The second is to *reuse computation*: persist and share caches so that

a receiver does not re-read what it has already read [3]. The third is to *send a latent message* in place of text, handing the receiver hidden states, a key–value cache or an embedding with no decoding into words. The training-free LatentMAS reports 70.8 to 83.7% fewer tokens and 4 to 4.3 times faster inference than text on nine benchmarks [10], and cache-to-cache reports a 2.5-fold reduction in latency together with 3.1 to 5.4% higher accuracy than text [11]. Other designs follow the same pattern [12, 13, 14, 15, 16, 17], and a recent framework organises them by what is transmitted and how the receiver absorbs it [18]. Which lever is appropriate is itself a decision that systems have begun to make adaptively, for instance by choosing between token and cache transmission according to bandwidth [19].

All of this work assumes that the agents are language models. The sender’s hidden state or cache is something another transformer can be trained to absorb. Yet the agents that sense the world are usually not transformers. A wearable or bedside classifier is a convolutional or recurrent network trained for classification, and nothing about its internal vector is shaped for a language model. The one heterogeneous exception we know of connects different families of vision–language models through a shared visual channel [20], but its senders are still language models. For a sender of this kind, it is open whether a latent channel is feasible, whether it is worth building, and what it costs in inspectability.

Neighbouring work does not close the question. Projecting a signal encoder into a language model’s embedding space is established [21, 22, 23, 24], but those systems are built to make a language model understand a signal and are judged on the accuracy of the result. They do not treat the projec-

tion as a channel whose cost is compared with a text channel on the same task. And comparisons of latent and text communication tend to use the text baseline that is easiest to beat. A prompt that lists every event verbatim is a poor engineering choice when the sender can filter, which is exactly the first lever above. Without a filtered baseline, a saving in tokens says little about whether a latent channel is worth building.

This paper compares the levers for a non-transformer sender. A small learned adapter maps the 32-dimensional context vector of a convolutional–recurrent heartbeat classifier, together with three scaled side inputs, to four virtual tokens per event in the input-embedding space of a frozen, 4-bit Gemma 4 E4B model. We compare it with three text interfaces, among them a sender-side filter, each also measured with prefix caching, on one task: report the most urgent triage tier among 1 to 50 consecutive heartbeats. Electrocardiography is the case study, a setting in which deep networks already match cardiologists [25] and language models encode clinical knowledge [26], so that pairing a sensing agent with a reasoning agent is a natural architecture. The task is deliberately a test of *communication fidelity*: the correct answer is a fixed rule applied to the sender’s outputs, so a wrong answer is a failure of the channel and not of clinical knowledge. A three-line rule also solves it, and we do not claim otherwise. The experiment measures what each interface costs and what it loses on the way, under a protocol fixed before the results existed. We ask three questions.

- **RQ1 (cost).** Does the adapter reduce the tokens, time and memory of the hand-off relative to text interfaces, and under which workloads?
- **RQ2 (accuracy).** Does it preserve the accuracy of the downstream

decision?

- **RQ3 (recoverability)**. How much of each event’s content can be recovered from the virtual tokens, so that the cost of opacity is measured instead of asserted?

The answers do not all favour the latent channel. It keeps prompt processing flat as events are added, while text grows by roughly 60 tokens per event, and it is more accurate than calibrated compact text at moderate window sizes. A text prompt that lists only the abnormal events, the filter, is more accurate than the adapter and equally fast for a single request. Its length depends on how many abnormal events a window contains, however, whereas the adapter’s depends only on the number of events, and this decides the comparison once requests are batched. We report the result in this form because it is what a system designer needs: a rule for which lever to pull.

The contributions are as follows.

1. A formulation of the interface choice for a non-transformer sender as a comparison of the three efficiency levers (filter, reuse, latent), with a text baseline that includes a sender-side filter.
2. A latent channel from such a sender into a frozen language model, extended from one event per call to fifty, in which the language model is untouched and only about 0.9 million adapter parameters are trained [TODO: verify parameter count at freeze].
3. A controlled, pre-registered evaluation of cost, accuracy and recoverability, including serving under batching and prefix caching, replication on a second sender architecture, and an external confirmatory test [TODO: insert result once the frozen run completes].

4. A recoverability protocol that decodes each event’s label, tier, heart rate, interval and abnormal-run status from its slot of virtual tokens, separating what the channel carries from what the receiver uses.
5. A rule for choosing between a filtered text interface and a latent one, with the pre-registration log and code to reproduce every table.

Section 2 places the work among the three levers and the literature on heterogeneous senders. Section 3 describes the sender, the adapter and the decoding scheme, and Section 4 the evaluation protocol. Sections 5 and 6 report and discuss the results, with limitations.

2. Related Work

We organise the related work by the three levers of communication efficiency introduced in Section 1, then turn to the two literatures that bear on a non-transformer sender: signal encoders connected to language models, and the effects that make long text prompts fail.

2.1. *Saying Less*

The first lever reduces what is sent. AgentPrune prunes redundant messages from the spatial–temporal message-passing graph of a multi-agent system in one shot and reports 28.1 to 72.8% fewer tokens across existing frameworks [1]. BANDMAS splits each message into semantic packets, such as evidence and requests, and transmits only those whose predicted contribution exceeds their cost, reducing application-layer bytes by 53.2 to 77.3% on three question-answering workloads [5]. Prompt-compression methods act inside the receiver’s context: they train a language model to compress a prompt

into a few learned tokens [6, 7], map a retrieved document to a single token [8], or prune the prompt with a small language model [9].

These methods share an assumption: the content is text that the sender could have chosen to write more briefly. For a perception agent that reports structured events, the equivalent is a filter that lists only the events that matter and counts the rest. We construct this baseline explicitly (Section 4). It is the standard against which a latent channel for such a sender has to be judged, and it is absent from prior comparisons of latent and text communication.

2.2. Reusing Computation and Choosing the Medium

The second lever avoids recomputing what the receiver has already read. On edge devices, persisting each agent’s cache in 4-bit form lets a multi-agent system resume with large reductions in time to first token [3]. These measures act on the cost of reading, not on the number of tokens a message needs. We therefore report prefix caching with every arm, to check that the ranking of interfaces does not depend on it.

A growing line of work treats the medium itself as a decision. Dai et al. select between token and key-value-cache transmission according to system conditions and allocate wireless bandwidth jointly, reducing end-to-end latency relative to either medium used alone [19]. A recent survey of communication-efficient agentic edge intelligence formalises what to send, when and to whom as a communication policy, and identifies communication as the primary design constraint at the edge [4]. Our selection rule (Section 6) is a policy of this kind, derived from measurements for a sender that these works do not consider.

2.3. Latent Communication Between Language-Model Agents

The third lever replaces the text message with a continuous one, and the designs differ in what is sent and how it is absorbed. CIPHER transmits the expectation of the vocabulary embeddings under the sender’s token distribution, so that the receiver sees the sender’s uncertainty as well as its sampled word [12]. LatentMAS shares last-layer hidden states and a latent working memory among agents without any training [10]. Cache-to-cache trains a projector and a fusion module that inject a source model’s key-value cache into a target model, across model families [11]. A latent cache-flow method transmits a compressed cache and equips the receiver with an adapter that translates and decompresses it [15]. In the bicameral model, two frozen language models are coupled through a trainable interface on their intermediate hidden states at every generation step [16]. Du et al. train a receiver-side adapter over compressed hidden states, report up to a 24-fold speed-up from compression, and present the work as a feasibility study [13]. Thought communication exchanges latent thoughts rather than words [14], and reusable latent building blocks have been proposed for multi-agent pipelines [17]. Liu et al. organise these designs by carrier and injection mechanism [18].

Reported benefits are lower cost and, in several cases, higher accuracy than text. What the methods share is a transformer sender. The transmitted object is a hidden state or a cache that a second transformer can be trained to absorb, and several of them require the two models to expose compatible internals or to be trained as a pair. The Vision Wormhole is the nearest heterogeneous case: it maps reasoning traces from different model families into a shared latent space through a universal codec and injects them into the

receiver’s visual pathway [20]. The heterogeneity is among vision–language models, and the sender is still a language model. A sender whose vector is the by-product of a classification loss on a physiological signal, with no language component, lies outside this literature.

A smaller body of work asks whether a latent channel is *used*. Zhang and Emu replace the message with another example’s message, with a compute-matched self-generated message or with nothing, and show that equal end-task accuracy can hide opposite mechanisms [27]. They build on the distinction between a message that depends on the sender’s state and a receiver whose behaviour depends on the message [28]. Our recoverability measurements answer a different question, namely what an auditor could read back from the message, and the two are complementary (Section 6).

2.4. Signal Encoders Connected to Language Models

Connecting a non-text encoder to a language model through a learned map is established practice. Visual instruction tuning projects image features into the embedding space [29], and Flamingo injects them through gated cross-attention [30]. For time series, Time-LLM keeps the language model frozen and reprograms input patches into its text-prototype space [21]. Babu et al. apply the same idea to multichannel EEG classification with a frozen language model [22]. SensorLM aligns wearable-sensor data and language by pretraining a CLIP- and CoCa-style model on 59.7 million hours of data [24]. In electrocardiography, ECG-Chat aligns a signal encoder with a language model for dialogue [23], ELF removes the separate encoder and lets the model read the signal directly [31], and zero-shot classification combines signal and clinical-text representations [32]. A benchmark of more than fifteen models

for fetal monitoring found that fine-tuned language models reached the best performance in most scenarios, at a higher computational cost than domain-specific architectures [33].

The frozen-model designs are the closest to ours in what they hold constant, but they differ in purpose. They aim to make a language model *understand* a signal and are judged on the accuracy of the resulting classifier or answer. None treats the projection as a message between two agents whose cost is compared with that of a text message on the same task, none compares against a sender-filtered text interface, and none reports what can be read back from the projected tokens. Our setting adds a constraint on the other side: the perception agent is trained for classification alone, with no language-aware objective, so the adapter is the only component that learns about the language model.

2.5. Why Long Text Prompts Degrade

Two documented effects explain why a text baseline weakens as events are added. Language models use information in the middle of a long context less well than information at its start or end [34], and the bias can be partly corrected by calibrating positional attention [35]. Separately, few-shot predictions are biased toward certain labels, which a calibration of the output scores corrects [36]. We find the label bias clearly in our text arms and weak, inconclusive evidence of a position effect (Section 5); following the first, we report text with a calibrated routine bias as the fair comparison instead of the default prompt.

Table 1: The interfaces compared, and the efficiency lever each implements. Prefix caching is applied to every arm as a second condition and is not a separate arm.

Arm	Lever	What crosses the interface
A-full	none (reference)	Full record of every event, as text
A-compact	none (baseline)	One compact line per event, as text
A-filtered	say less	Abnormal events only, normal events as a count
Adapter (r4)	send latent	Four virtual tokens per event

2.6. Gap

Saying less, reusing computation and sending latent messages have each been studied for agents that are language models. What has not been shown is a controlled, same-task comparison of these levers for a *non-transformer sender*, in which the text baseline *includes a sender-side filter*, the receiver is frozen throughout, cost is measured under serving conditions, and the information a latent channel preserves is measured and not assumed. The remainder of the paper supplies that comparison.

3. System

Figure 1 shows the pipeline. A perception agent classifies each heartbeat and emits an event. The events of a window cross an interface to a reasoning agent, which returns a structured triage decision. The interface is the only thing that varies between arms; both agents are identical across arms. Each arm implements one of the efficiency levers of Section 1 (Table 1), so that the comparison is a comparison of levers on a fixed sender and a fixed receiver.

[TODO: Figure 1: pipeline; perception agent $\rightarrow$ interface (text or adapter) $\rightarrow$ frozen reasoning agent. Redraw from reports/figures/fig1_pipeline.png.]

Figure 1: System overview. The highlighted block, the adapter, is the only component trained for this work.

3.1. Task

Each window contains N consecutive heartbeats from one recording, with $N \in \{1, 5, 10, 20, 50\}$. The reasoning agent must return the most urgent triage tier in the window and say which beat set it. The tier of a single beat follows a fixed escalation rule applied to the perception agent’s event: *urgent* if the event requires urgent review (a run of three or more consecutive abnormal beats, or a single ventricular or fusion beat classified with confidence of at least 0.85), *routine* if the beat is classified as normal, and *priority* otherwise. The reference tier of a window is the maximum over its beats. The rule is applied to the perception agent’s *predictions*, not to the annotations, because the question being tested is whether the channel delivers the sender’s output faithfully. Agreement with the annotations is reported separately (Section 5). We treat the task as a communication-fidelity benchmark and not as a clinical reasoning task: a fixed rule over the perception outputs solves it, so it cannot show that a language model adds clinical judgement.

3.2. Perception Agent

The perception agent is a convolutional–recurrent classifier trained only on the five-class AAMI EC57 beat task [37]. A 360-sample window (1 s at 360 Hz) passes through three one-dimensional convolutional blocks (32,

64 and 128 channels, with batch normalisation, ReLU and max-pooling), a bidirectional LSTM with hidden size 64, and a unidirectional context LSTM with hidden size 32. The output of the context LSTM, a 32-dimensional *context vector*, feeds the classification head and is the only representation passed downstream. There is no pretraining, no contrastive objective and no auxiliary loss.

The original design saw a single window and no timing, yet the text interface carried the RR interval. A five-feature RR branch (previous and next RR interval, local mean RR and two ratios) was therefore added to the context stage. Across ten training seeds it improved overall accuracy from 0.854 to 0.926 ± 0.019 , ventricular sensitivity from 77% to $93\% \pm 3\%$ and the class recoverability of the context vector from 69.6% to $81.2\% \pm 5.0\%$, at the cost of one beat of latency. Fusion-beat detection stayed poor ($5\% \pm 7\%$ sensitivity). All adapters were trained on seed 0 of this encoder, which ranked third, fourth and seventh of ten on accuracy, supraventricular sensitivity and recoverability and was designated representative by a rule fixed beforehand.

A second, purely convolutional sender (residual one-dimensional convolutions without recurrence, the same RR branch and the same 32-dimensional vector) tests whether the channel depends on the first sender’s architecture (Section 5).

3.3. Reasoning Agent

The reasoning agent is Gemma 4 E4B, loaded in 4-bit NF4 quantisation and frozen in every arm. Its weights are never updated, either by adapter training or by calibration. The model uses per-layer embeddings, a lookup keyed by token identity, which virtual tokens do not have. Virtual positions

therefore receive the lookup of a padding token. This is an architectural cost of the latent channel and applies to every adapter result.

3.4. Adapter: the Latent Channel

The adapter is the third lever. It must satisfy three requirements that follow from the setting: the sender is not changed and is not a transformer, the receiver is not changed, and the message must have a length that depends only on the number of events. A linear map from the sender’s own vector meets all three, and keeps each event’s tokens a function of that event alone. For event i with context vector $\mathbf{z}_i \in \mathbb{R}^{32}$ and three scaled side inputs $\mathbf{s}_i \in \mathbb{R}^3$ (heart rate, RR interval and abnormal-run length, each clipped and standardised), the adapter forms $\mathbf{x}_i = [\mathbf{z}_i; \mathbf{s}_i] \in \mathbb{R}^{35}$ and produces $k = 4$ virtual tokens

$$\mathbf{T}_i = \gamma \cdot \text{norm}(\mathbf{W}\mathbf{x}_i + \mathbf{b} + \mathbf{P}_i) \in \mathbb{R}^{4 \times 2560}, \quad (1)$$

where $\mathbf{W}$ and $\mathbf{b}$ define one linear map from 35 to 4×2560 dimensions, $\mathbf{P}_i \in \mathbb{R}^{4 \times 2560}$ is a learned embedding for event position i (up to 50 positions), norm rescales each token to unit length, and γ is one learnable scalar initialised to the median norm of the embeddings the model computes for real tokens. The normalisation fixes a failure observed early in development: without it, the norm of the virtual tokens drifted far from that of real tokens, and many of them together broke generation. The tokens of the window are the concatenation of the $\mathbf{T}_i$ in event order and replace the text of the events in the prompt. Prompt length therefore depends only on N .

Because the map is linear and acts on one event at a time, slot i is a function of event i alone, so each event’s content can be audited independently

(Section 5). The map has rank at most 35, so the adapter cannot introduce information absent from its input. About 0.9 million parameters are trained, against 4 billion in the frozen model [TODO: confirm parameter count from the checkpoint at freeze].

Training minimises a weighted next-token loss on the structured answer, with the tier tokens weighted fourfold, using AdamW with warm-up and cosine decay, a batch size of one window, and early stopping on a validation set of held-out recordings. The final recipe, r4, differs from the earlier recipe r3 in two ways, both pre-registered before training. First, 72 training and 23 validation windows were added in which a normal beat is classified with confidence below 0.8 (*hard negatives*), because r3 raised false alarms on exactly such windows. Second, the three side inputs were added, because heart rate, interval and run length were present in the text interface but could not be recovered from the 32-dimensional vector. The two changes were made together; their separate effects are not identified (Section 6).

3.5. Text Interfaces

All text arms use the same scaffold, instructions and decoder as the adapter arm. They differ only in how the events are written into the prompt. A-compact and A-full list every event and so involve no effort to say less; A-filtered is the sender-side filter, the first lever.

- **A-compact:** one line per event with class, confidence, run length and signal quality. This is the strongest text arm that lists every event and is the primary comparator.
- **A-full:** the full JSON record of each event, as in the earlier single-event

study, reported only for cost.

- **A-filtered:** only the non-normal events, with their position in the window, and the number of normal events as a count. No task information is lost: the filtered window yields the reference tier on all 1,195 test windows.

A text arm that lists every event is also reported under a *calibrated* threshold (Section 4).

3.6. Schema-Constrained Decoding

Free generation produced malformed JSON in 9 to 20% of answers across arms, through misnamed fields and loops. Every arm therefore uses the same constrained decoder. The structure of the answer is forced, the tier is restricted to the three valid words and chosen by the model’s own scores, and free-text fields cannot close or break the structure. Every arm then produces a valid answer on every window. A check of 380 windows confirmed that the decoder changes no tier.

4. Evaluation Protocol

The protocol is designed so that a difference between arms can be attributed to the lever and not to the prompt, the model or the measurement. It fixes the sender, the receiver, the scaffold and the decoder; it measures cost where cost arises, on the prompt side and under serving conditions; and it scores accuracy with a rule agreed before the results existed.

4.1. Pre-Registration and Deviations

The analysis plan, with its endpoints, comparators, statistics and decision rules, was committed before any Phase 1 run; the commit timestamp is the record. Every later change of design was written into a numbered log, with its date and reason, before the data it affected were produced, and earlier text was never edited. Analyses run after seeing results are labelled exploratory. The log contains [TODO: number of deviations at freeze] entries and is part of the supplementary release. We report where a deviation altered an outcome, and where an analysis registered in advance gave an answer that we do not read literally (Section 5).

4.2. Data

Training and primary test. The MIT-BIH Arrhythmia Database [38] was used with the inter-patient split of de Chazal et al. [39]: 22 records for training and 22 for testing, with no patient in both, after the four paced records were excluded. Of the training records, 19 were used for training and three held out entirely for validation, so that no tuning decision touched a test patient. Beats were extracted as 360-sample windows centred on the annotated R peak and standardised per window, and annotations were mapped to the five AAMI classes. The test set contains no unclassifiable beats, a property of the split.

Chronological replay. The abnormal-run count is state carried across beats of a record. Events were therefore produced by replaying each record in order from its start through the frozen perception agent, so that the run count and RR interval of every window are those a deployed system would have computed.

Test windows. A fixed sampler (seed 0) draws windows from the test records, stratified by reference tier and by the class of the window’s most urgent beat, at most three per record per cell, so that urgent and priority windows are not dominated by one class. The primary set has 1,195 windows from 22 patients, of which 400 are drawn at natural prevalence to measure false alarms. Cells that cannot be filled are left short and the achieved counts reported. [TODO: Table of windows per N and tier.]

External confirmatory data. The St Petersburg INCART 12-lead arrhythmia database [40] (75 half-hour recordings from 32 patients) was fixed as the confirmatory test before it was downloaded or inspected. Lead II is resampled from 257 to 360 Hz and converted with the same procedure, and no INCART data are used for training, tuning or prompt design. The adapter, the text prompts and the calibration biases are frozen and hashed in a manifest before the run. [TODO: Report INCART results when the frozen run completes.]

4.3. Comparators and Calibration

The comparators are A-compact, A-full (cost only) and A-filtered (Table 1). Reusing computation, the second lever, is not an arm: prefix caching is switched on and off for every arm in the serving measurements, so that the ranking of the other levers can be checked under it. A text model with every event listed is biased toward answering “routine” on long prompts, an effect known from few-shot prompting [36]. To avoid a strawman we report each text arm both under default decoding and with a bias added to the routine score at the tier step, chosen on the held-out validation windows only and then frozen. The calibrated arm is the fair comparison and is used for every

inferential statement.

4.4. Endpoints and Statistics

Accuracy. The primary endpoint is balanced accuracy, the mean recall over the tiers present, so that the rare urgent tier carries the same weight as the common routine tier. Arms are compared on the same windows (paired). Because windows from one patient are correlated, confidence intervals use a patient-cluster bootstrap with 20,000 resamples and percentile 95% limits. The primary question for the adapter against calibrated text is non-inferiority with a margin of -0.05 , tested first, followed by superiority where non-inferiority holds, in a fixed order at $\alpha = 0.05$. Adapter results are the mean of three training seeds, and seed ranges are reported.

False alarms. The share of routine windows at natural prevalence that the arm escalates to priority or urgent.

Cost. Token counts of the prompt; time to first token (prompt processing) and time to the tier token (the decision), at batch size one with arm order rotated; context memory; and GPU energy. We report time to the decision, and not time to the full answer, because answer length differs across arms and decode speed is identical (about 160 ms per token); including it would attribute to the interface a difference that belongs to the length of the justification. Cost medians use a paired bootstrap of the relative difference on 21 windows per N . [TODO: Check the count against the final timing files.]

Serving. Throughput and energy per decision were measured with batches of one, four and eight, repeated three times, and with prefix caching on and off. The framework was capped at 90% of the card’s memory so that a batch

that does not fit is recorded as such instead of being silently paged into system memory, which would measure paging and not the model.

Recoverability. For each event, a decoder is trained on the training records to predict label, tier, heart rate, RR interval and abnormal-run status from the virtual tokens of a given slot, and tested on the test records. The same decoder family is trained on the 32-dimensional input and on the 35-dimensional input including side inputs, so that the loss attributable to the channel is the gap between the two. Decoders are fitted per slot (slots 0, 9, 19 and 49). The registered decoder is a small multilayer perceptron; a linear decoder is fitted alongside it for comparison. Run status is scored as balanced accuracy for a run of three or more beats, because R^2 is ill-posed for the heavy-tailed run length on this split (a change registered before any data from it).

4.5. Implementation and Reproducibility

All experiments ran on one workstation with an NVIDIA RTX 5070 (12 GB), Python 3.13 and HuggingFace `transformers` with `bitsandbytes`. Production serving engines do not accept continuous input embeddings, so absolute latencies are higher than an optimised engine would give; because every arm shares this, only relative comparisons are interpretable. Every run writes its git commit, checkpoint hashes, seed and library versions. Adapter training checkpoints are resumable and bit-identical on resume, and one training run was paused and resumed without effect on the result. Numbers reach the manuscript only through a results ledger generated from the result files. [TODO: Add repository and archive DOI after the freeze.]

5. Results

All numbers in this section come from the pre-registered DS2 evaluation (1,195 windows, 22 patients) unless stated otherwise, and are generated from the results ledger [TODO: generate the tables from the ledger before submission]. Adapter values are means of three training seeds. Intervals are 95% patient-cluster bootstrap limits.

The results answer one question for each research question: what does the latent lever cost relative to the others (RQ1), what does it give up or gain in the decision (RQ2), and what can still be read back from the message (RQ3). Throughout, the relevant comparison is against the strongest text arm that is available for the question being asked: calibrated compact text for accuracy against listing every event, and filtered text, the first lever, for every claim about whether the latent channel is worth building. Section 6 brings the three answers together in a selection rule.

5.1. RQ1: Cost of the Interface

At a single event the adapter has nothing to save. Its four tokens replace about 57 tokens of compact text, but the fixed scaffold dominates the prompt (end-to-end lengths of 486 and 458 tokens for the two arms in the single-event study), and time to first token is unchanged. The advantage appears only when a prompt carries many events (Table 2). Text grows by about 60 tokens per event whereas the adapter adds four, so the adapter’s prompt-processing time stays near 183 ms from 1 to 50 events while the time of compact text rises to 845 ms. Time to the tier token falls by a smaller factor because the decision includes the generation of the first answer tokens, which is the same

Table 2: Cost of the adapter relative to compact text (batch one, median over 21 windows per N).

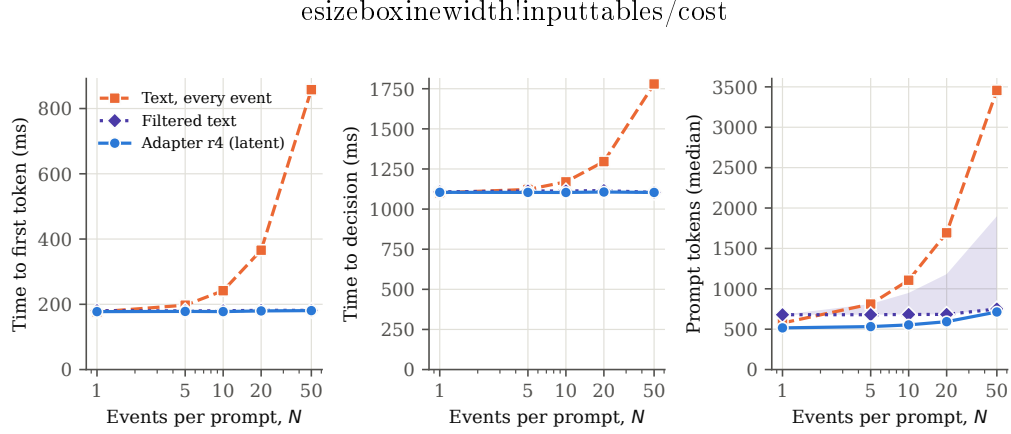


Figure 2: Cost of the three interfaces against the number of events per prompt (batch one, median of 21 windows per N). The shaded band in the right panel is the 90th percentile of the filtered prompt: its length depends on content, whereas the adapter’s depends only on N . Time to decision is almost identical for filtered text and the adapter.

in both arms.

Serving. Single-request timing flatters neither arm because nothing is reused. Under batching, text that lists every event could not be served at 50 events, nor at a batch of eight at any $N \geq 10$, on the 12 GB card, whereas the adapter fitted every configuration with a peak of 10.0 GB. Against A-filtered, the adapter was within about 5% for single requests and was ahead once requests were batched, with 10 to 30% more decisions per second and 12 to 28% less energy per decision (Table 3). Filtered text fitted only some batches of eight, because its length depends on how many abnormal beats a window contains; the filtered figures at batch eight come only from the

Table 3: Decisions per second and GPU energy per decision when requests are batched (adapter: r4, seed 101). “Does not fit”: the batch exceeds the 12 GB memory cap.

esizeboxinewidth!inputtables/serving

Table 4: Balanced accuracy on class-balanced DS2 windows, and adapter minus calibrated text with 95% patient-cluster intervals. The adapter is superior at $N = 10$ and 20 (interval above zero) and non-inferior at $N = 5$ and 50 (lower limit above the margin of -0.05).

N	Text	Text cal.	Adapter r4	Adapter – cal. text [95% CI]
1	0.97	0.97	0.80	—
5	0.73	0.79	0.81	+0.03 [−0.04, +0.10]
10	0.56	0.73	0.83	+0.10 [+0.03, +0.17]
20	0.48	0.73	0.83	+0.10 [+0.01, +0.18]
50	0.54	0.76	0.84	+0.08 [−0.02, +0.17]

batches that fit, which are its shortest, so they favour filtered text. Prefix caching shortened every arm’s prefill and did not change the ranking.

5.2. RQ2: Accuracy

Against text that lists every event. Default compact text falls from 0.97 balanced accuracy at one event to 0.48 to 0.56 at 10 to 50 events. Calibration recovers part of this (0.73 to 0.76) and is the fair comparison. The adapter (r4) holds 0.81 to 0.84 at 5 to 50 events (Table 4). Against calibrated text, the adapter was superior at 10 and 20 events (both +0.10, intervals excluding zero) and non-inferior at 5 and 50 events. At a single event, text was better (0.97 against 0.80), as expected from the cost analysis.

Why text with every event fails. Two mechanisms account for most of the loss (Figure 5). Text misses urgent windows caused by a single high-

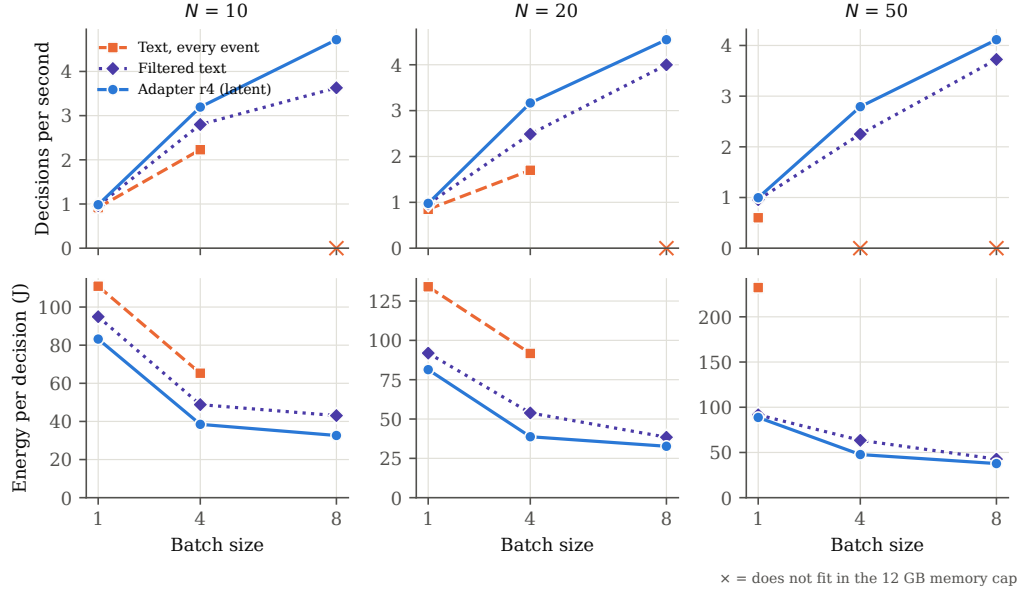


Figure 3: Throughput (top) and GPU energy per decision (bottom) when requests are batched, for 10, 20 and 50 events per prompt (adapter: recipe r4, seed 101). A cross marks a configuration that did not fit in the memory cap of the 12 GB card. Filtered text at batch eight is plotted from the batches that fit, which are its shortest prompts.

confidence ventricular or fusion beat (a numeric threshold of 0.85 buried in a long list) far more often than windows caused by a run of abnormal beats, and the first of these is the clearest effect: text recalls 0.17 of such windows against 0.82 for the adapter. The position of the deciding beat points in the direction reported for long contexts [34, 35]: text accuracy is lowest when that beat is in the middle third of the window (0.19, against 0.30 in the first and 0.38 in the last third). The evidence is weak, however. The middle group has 73 windows, and its interval overlaps those of the other thirds (right panel of Figure 5). The adapter varies less across positions (0.74 to 0.78) but is not flat, with a similar small dip in the middle third. We therefore report

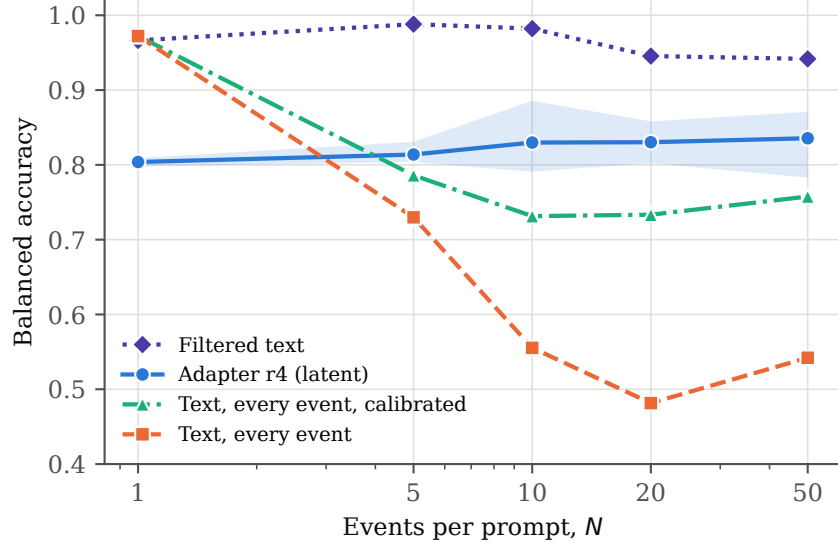


Figure 4: Balanced accuracy against the number of events per prompt on the class-balanced test windows. The shaded band is the range of the three adapter seeds. Filtered text is the most accurate arm; text with every event listed degrades as events are added and calibration recovers only part of the loss.

a position effect as suggested for text and not as established, and as not distinguishable from the adapter’s.

False alarms. The previous recipe escalated about 11% of routine windows, concentrated on windows with a normal beat of low classification confidence. Recipe r4 reduced the rate by about two thirds at every seed, to 3.6% overall (0% to 2.5% at 50 events). Text raised none, because its error is the opposite one: it under-escalates urgent windows to priority. The adapter never called an urgent window routine in the 50-event confusion matrix.

What the final recipe’s gain comes from. Recipe r4 changed two things at once, hard-negative routine windows in training and three side

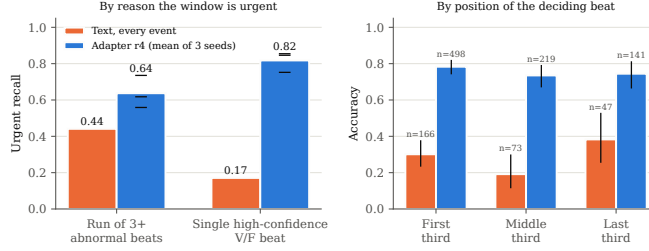


Figure 5: Urgent recall by the reason a window is urgent. Text with every event listed misses windows made urgent by a single high-confidence ventricular or fusion beat far more often than windows made urgent by a run; the adapter does not show this gap. Bars for the adapter are the mean of three seeds, with each seed’s value marked. Right: accuracy on non-routine windows with $N \geq 10$ by the position of the window’s most urgent beat; whiskers are 95% Wilson intervals, and the adapter counts are windows times three seeds. This panel is exploratory (not pre-registered) and is computed by `scripts/position_effect.py`.

inputs per event (heart rate, interval and run length), so we retrained it without the side inputs, three seeds, everything else identical (Table 5). The side inputs account for most of the improvement: accuracy rose at every N (by 0.03 to 0.05, with the interval excluding zero at 50 events) and false alarms fell from 8.6% to 3.6% in every seed. Hard negatives alone moved accuracy by about 0.01 and false alarms from 10.6% to 8.6%. The false-alarm reduction therefore comes mainly from giving the receiver the context to tell a low-confidence normal beat from a real event, not from the extra training examples.

Against text that lists only abnormal events. A-filtered was more accurate than the adapter at every N (Table 6), raised no false alarms, and for a single request was equal in time to first token and to decision within

Table 5: Ablation of recipe r4 (three seeds each): balanced accuracy, false alarms (FA) on routine windows, and the two contrasts with patient-cluster 95% intervals. HN: hard-negative routine windows in training; SI: side inputs (heart rate, interval, run length). The SI contrast is clean (same windows and schedule); the HN contrast is approximate, because r3 also used fewer training windows and a proportionally shorter schedule.

Recipe	$N=5$	$N=10$	$N=20$	$N=50$	FA
r3 (neither)	0.794	0.772	0.795	0.770	10.6%
HN only	0.788	0.786	0.802	0.781	8.6%
r4 (HN + SI)	0.814	0.830	0.830	0.836	3.6%
SI: r4 – HN only	+0.03 [−0.02, +0.06]	+0.04 [−0.01, +0.11]	+0.03 [−0.01, +0.07]	+0.05 [+0.01, +0.11]	
HN: HN only – r3	−0.01 [−0.04, +0.03]	+0.01 [−0.02, +0.05]	+0.01 [−0.04, +0.05]	+0.01 [−0.03, +0.05]	

1.5%. Its cost depends on content: each listed abnormal beat adds about 68 tokens, so at 50 events its prompt is longer than the adapter’s in half of the windows. The pre-registered decision rule for this comparison anticipated a filtered prompt that was cheaper but less accurate, and applied literally it returned “advantage holds” because the filtered prompt turned out to be longer. We do not read it that way. Against filtered text, on this task, the latent interface offers no accuracy advantage and no latency advantage for a single request.

Against the annotations. Scored against the database annotations instead of the sender’s own rule, the encoder’s rule has sensitivity of 0.92 to 0.94 and specificity of only 0.46 to 0.52 for a truly abnormal beat in the window. The clinical ceiling is therefore set by perception and not by the interface, and the adapter tracked the reference (sensitivity 0.89 to 0.98 for the earlier recipe) while default text missed more abnormal windows (0.67 to 0.71 at 10 events).

Second sender. The replication with a convolutional sender (validation

Table 6: Adapter against filtered text (default decoding) with patient-cluster intervals, and median prompt tokens (adapter / filtered).

N	Adapter r4	Filtered	Difference [95% CI]	Tokens
1	0.804	0.966	—	516 / 679
5	0.814	0.988	−0.17 [−0.22, −0.13]	532 / 679
10	0.830	0.982	−0.15 [−0.19, −0.11]	553 / 682
20	0.830	0.945	−0.12 [−0.16, −0.07]	593 / 683
50	0.836	0.942	−0.11 [−0.17, −0.05]	713 / 750

accuracy 0.955 against a gate of 0.85) reproduced the pattern (Table 7). Text with every event listed fell as events were added while the adapter stayed at 0.77 to 0.82; the adapter was superior to calibrated text at 10 events and non-inferior at 20 and 50, but not at 5; false alarms were 1.3 to 2.1%. A-filtered was again the most accurate arm except at 50 events, where it fell to 0.80 and the difference was inconclusive (−0.01 [−0.09, +0.08]). An exploratory look at those windows (40 per sender, so indicative only) suggests that this sender’s urgent windows carry sparser, more borderline evidence, often one urgent beat among about 13 listed abnormal beats with a deciding confidence just above 0.85. Filtered text depends on how the sender’s evidence is distributed, whereas the adapter did not change much.

External confirmatory test. [TODO: INCART: H1 to H4 results from the frozen run (non-inferiority at $N = 10, 20, 50$; superiority at 10 and 20; non-inferiority against filtered text; time to first token). Report a failure of any hypothesis as plainly as a pass.]

Table 7: Second sender (ResNet1D-RR): balanced accuracy on its own DS2 windows and adapter minus calibrated text (non-inferiority margin -0.05).

N	Adapter	Text cal.	Filtered	Adapter – cal. text [95% CI]
5	0.770	0.787	0.959	$-0.02 [-0.11, +0.08]$
10	0.823	0.732	0.929	$+0.09 [+0.02, +0.16]$
20	0.780	0.733	0.907	$+0.05 [-0.03, +0.13]$
50	0.794	0.733	0.808	$+0.06 [-0.02, +0.14]$

5.3. RQ3: What Can Be Recovered

Single events. Decoders trained on the training records and applied to the test records recovered the predicted beat label (99.5%) and tier (95%) from the virtual tokens as well as from the 32-dimensional input, so the adapter loses nothing the vector holds.

Every position. For slots 0, 9, 19 and 49 of a 50-event window, label, tier and urgency decoded at the level of the input vector. Decoders are slot-specific: a decoder trained on slot 0 applied at slot 49 fell to 0.64 to 0.91, so an auditor needs a position-aware decoder.

What the vector never carried. The 32-dimensional context vector does not contain heart rate, interval or run length (R^2 about 0.3), so the first recipe could not carry them either. With the side inputs of r4 they are recoverable from the tokens (Table 8). The decoder registered in advance was a small multilayer perceptron, and with it heart rate (R^2 0.59 to 0.78) and interval (0.65 to 0.77) fall well short of the input (0.98 and 0.95), while the presence of a run of three abnormal beats is recovered at 0.93 to 0.98 balanced accuracy (0.53 to 0.61 for the first recipe, near chance). A linear decoder, fitted in the same analysis, recovers much more of the numeric fields

Table 8: Recoverability of each event’s fields from its virtual tokens (recipe r4, main sender), decoders trained on the training records and applied to the test records. Token columns give the range over slots 0 and 49 and three seeds. The multilayer perceptron was the registered decoder; the linear decoder is reported for comparison.

Field	Input, MLP	Tokens, MLP	Input, linear	Tokens, linear
Beat label (bal. acc.)	0.99	0.95–0.98	0.99	0.99–1.00
Tier (bal. acc.)	0.95	0.92–0.95	0.91	0.90–0.94
Urgency flag (bal. acc.)	0.98	0.97–0.98	0.94	0.94–0.95
Heart rate (R^2)	0.98	0.59–0.78	1.00	0.95–0.97
RR interval (R^2)	0.95	0.65–0.77	0.98	0.92–0.96
Run ≥ 3 (bal. acc.)	0.98	0.93–0.98	0.95	1.00–1.00

from the same tokens (heart rate 0.95 to 0.97, interval 0.92 to 0.96), which indicates that the multilayer decoder understates what the tokens hold, and that the information is held in a nearly linear form. We report both because the registered figure is the conservative one, and treat the linear figures as the better estimate of what an auditor could recover. On the second sender, heart rate and interval are recovered from every token slot at R^2 0.89 to 0.94 with the registered decoder. Signal quality is not carried by either sender.

Carried and used. The ablation shows both directions. Without the side inputs, heart rate, interval and run length are not recoverable from the tokens at all (R^2 below zero; run ≥ 3 at 0.55, near chance); with them they are, and the receiver uses them, since they account for most of the accuracy gain and the false-alarm reduction (Table 5). Use is partial, however: urgent recall on windows made urgent by a run of three or more abnormal beats rose only from 0.61 to 0.64 when run length entered the tokens, against 0.44

for text.

Pre-generation check. A score computed from the context vector alone predicts the adapter’s wrong answers (AUROC 0.89 to 0.97) and flags noise and powerline interference (0.99), which gives an inspection that does not need to read the tokens.

Compression. We retrained the adapter with one, two and eight virtual tokens per event in place of four (Table 9, Figure 6). A first sweep with one seed per setting suggested that a single token might suffice, so one and two tokens were repeated with three seeds each, under a rule registered before training: a setting is free of accuracy cost only if it is non-inferior to four tokens (margin -0.05) at every N . Neither passed. With one token, one run in three failed to learn the task (0.49 to 0.55 at 5 to 50 events) while the other two matched four tokens, so the three-seed mean was 0.09 to 0.11 lower. With two tokens accuracy was within about 0.01 of four tokens at 5 to 20 events and 0.04 lower at 50, non-inferior only at 20 events, and false alarms were higher in every run (8.6% on average against 3.6%). Eight tokens, a single run, was no better than four. Each event’s label, heart rate and run status remained recoverable from a single token, including from the one-token run that failed the task (label 0.96 to 0.99, heart-rate R^2 about 0.9 with the registered decoder). Compression below four tokens per event therefore preserved what the tokens carry but made the receiver’s use of it unreliable, and four tokens was the smallest setting at which every run was reliable. The prompt saving it would buy is modest, because the scaffold dominates: 563 tokens at one token per event against 713 at four, at 50 events.

Table 9: Virtual tokens per event: three seeds for $k = 1, 2$ and 4 (means), one run for $k = 8$. Tokens: prompt length at 50 events. $k - 4$: difference from $k = 4$ at the least favourable N (50 in both cases), with its patient-cluster 95% interval; NI: number of N (of 5, 10, 20, 50) at which k is non-inferior (margin -0.05). FA: false alarms on routine windows (seed ranges in the text). Label and HR R^2 : recoverability from slot 0 with the registered decoder (multilayer perceptron); the negative R^2 at $k = 8$ is likely a decoder limit (32 principal components of a 20,480-dimensional slot).

k	Tokens	Bal. acc., $N = 5/10/20/50$	$k - 4$ [95% CI]	NI	FA	Label	HR R^2
1	563	0.70 / 0.72 / 0.75 / 0.73	$-0.11 [-0.16, -0.06]$	0/4	6.3%	0.97	0.87
2	613	0.80 / 0.82 / 0.83 / 0.80	$-0.04 [-0.13, +0.03]$	1/4	8.6%	0.95	0.81
4	713	0.81 / 0.83 / 0.83 / 0.84	ref.	–	3.6%	0.96	0.71
8	913	0.77 / 0.80 / 0.78 / 0.80	1 run	–	6.1%	0.95	-0.62

6. Discussion

6.1. Choosing a Communication Interface

The three research questions give three answers, and they do not point to the same arm. Table 10 sets the arms side by side on the criteria a system designer would use. Reading across it, no interface dominates: the filter wins on accuracy and readability, the latent channel on predictability of cost under load, and listing every event on nothing except simplicity at small N .

From the table we draw a selection rule for the task studied, with one part stated as a hypothesis.

- **The receiver’s question is fixed and known when the sender writes its message:** filter on the sender side and send text. The filter was the most accurate arm at almost every N on both senders, raised no false alarms, matched the adapter for a single request, and can be read by a person without any decoder.

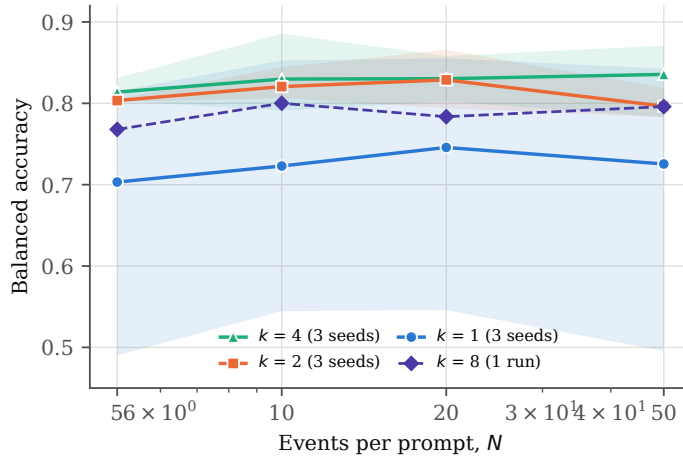


Figure 6: Balanced accuracy against the number of events for $k = 1, 2$ and 4 virtual tokens per event (lines: mean of three seeds; bands: seed range) and $k = 8$ (one run). The wide band at $k = 1$ is one failed run.

- **Many events must be communicated at a predictable cost, or requests are batched on constrained hardware:** use the latent channel. Its length depends only on N , so memory and batch size can be planned in advance, and in the batched measurements it delivered 10 to 30% more decisions per second at 12 to 28% less energy per decision than filtered text.
- **The question is not known when the sender encodes** (hypothesis, not tested here): a filter cannot discard what it cannot rank, whereas the latent channel carries information about every event. Our task has one question, so the experiment cannot show that the extra information would be used by a different one. A study that varies the question over fixed virtual tokens is the direct test, and we leave it to

Table 10: Interfaces compared on the criteria that decide the choice, from the measurements in Section 5. “Content-dependent” means that the cost varies with how many abnormal events a window contains.

Criterion	Text, every event	Filtered text (say less)	Adapter (latent)
Accuracy ($N \geq 10$)	Lowest, even calibrated	Highest	Between; above calibrated text at $N = 10, 20$
Prompt length	Grows with N	Content-dependent	Depends only on N
Single-request latency	Grows with N	Equal to adapter within about 5%	Flat in N
Batched throughput and energy	Does not fit at batch 8 or at $N = 50$	Between; fits only some batches	Highest; fits every configuration
Memory planning	Impossible at large N	Content-dependent	Fixed per event
False alarms	None	None	1 to 4% of routine windows
Auditability	Readable	Readable; normal events dropped	Needs a position-aware decoder

future work.

Against text that lists every event, the adapter wins on cost and, at moderate window sizes, on accuracy, but that is a comparison with a prompt no careful engineer would send.

This rule is a communication policy of the kind that recent edge-intelligence work formalises, in which the system decides what to send and in what form according to conditions [4, 19]. Existing policies choose between text and a

transformer’s cache. Our results add a third option for senders that are not transformers and show that, for them, the choice turns on the shape of the workload and the serving regime more than on accuracy.

6.2. Why the Filter Wins on Accuracy and the Adapter Wins Under Load

The two results have different causes. The filter is accurate because it uses the sender’s own labels to remove exactly the events that cannot change the answer, leaving a short list the receiver can reason over; its accuracy therefore depends on how the sender’s evidence is distributed, as the second sender shows, where filtered text fell at 50 events when urgent windows rested on a single borderline beat among many. The adapter is less accurate, and we offer two plausible reasons that this study does not separate: the receiver has to learn from a small training set to read tokens that were never part of its vocabulary, and the linear map from a classification-trained vector carries less than the sender’s text fields do (the side inputs of the final recipe were added to close that second gap; the ablation shows they drive the final recipe’s gain, and the run-length result shows the gap only partly closed). It wins under load for a structural reason: the filter’s length is a function of content, so batches of long prompts do not fit, whereas the adapter’s length is a function of N alone. The first is a property of the sender’s output and the second a property of the interface, and a designer who knows the workload can read off which regime applies.

6.3. What the Result Means for Auditability

The standing objection to a latent channel is that no one can read it. We find that much can be read back by training decoders on the tokens: label,

tier and abnormal-run status at the level of the input, and heart rate and interval nearly so with a linear decoder, at every slot tested. What an auditor needs is a position-aware decoder per slot, and what the channel does not carry (signal quality) is identified by the same method. But recoverable is not the same as used. The compression study shows this most clearly: at one virtual token per event every field remained recoverable, including from a run that failed the task, so a recoverability audit can certify what the channel carries but not that the receiver relies on it. The run-length experiment shows a field that reaches the language model without being used reliably, and a pre-generation score computed from the context vector gives an inspection that does not depend on the tokens at all. Zhang and Emu show, for latent channels between language models, that equal end-task accuracy can arise from opposite mechanisms and that a message must be replaced to know whether it matters [27]. The same discipline applies here, and the null and shuffle controls registered in the analysis plan are the receiver-side half of the measurement. [TODO: State their results here if they are part of the final analysis; otherwise list as future work.]

6.4. Limitations

- **The task is a fixed rule.** The reference answer is a three-line rule over the perception outputs, so a language model is not needed for the answer. The study measures a channel, not reasoning, and says so. It does not show that the language model contributes judgement.
- **One receiving model.** All results use Gemma 4 E4B in 4-bit quantisation. A second receiver, of a different family, is future work, and the

adapter is specific to one embedding space.

- **One database for tuning, one external test.** The primary test set informed several design iterations. Each was logged before the data it affected, but the set is no longer untouched. The external confirmatory test on INCART was fixed for this reason [TODO: update with the result].
- **Training variance.** Outcomes vary between training seeds (validation balanced accuracy 0.72 to 0.82 for r4, and one failed run at one token per event), so every adapter result is reported over three seeds; the eight-token setting is a single run.
- **Perception quality.** Specificity of the encoder’s rule against the annotations is 0.46 to 0.52, and fusion beats are almost never detected. The ceiling on usefulness is set upstream of the interface.
- **Zero-shot text baseline.** The adapter was trained on the task while the text arms are zero-shot under a frozen-model constraint. A trained text baseline would narrow the accuracy gap and was not run.
- **Fairness of calibration.** Calibration chose a single bias on the routine score. A richer calibration could further improve text with every event listed.
- **Hardware and software.** Timing was measured on one consumer GPU through a general-purpose framework, because production engines do not accept continuous embeddings; only relative results are

meaningful. Network transfer between agents was not measured, because both agents ran on one machine.

- **Clinical use.** The system is a research prototype. No clinician rated its outputs, and nothing here supports clinical deployment.

6.5. Implications for System Design

Three practical points follow. First, the value of a latent channel is a property of the workload and not of the channel: report cost against the number of events and under batching, and include a filtered text baseline, or the comparison says little. Second, a fixed per-event footprint changes how a system is provisioned even where accuracy is equal, because batch size no longer depends on what the sender has to say. Third, inspectability can be measured and not merely asserted: train decoders on the message and report what they recover, per slot and per field.

7. Conclusion

For a perception agent that is not a transformer, there is a choice of how to talk to a frozen language model, and it is not settled by accuracy alone. A sender-side filter is the more accurate and more readable interface when the receiver’s question is known in advance. A four-token-per-event linear adapter gives a latent channel whose cost is fixed per event, which is more accurate than calibrated compact text at moderate window sizes and, under batched serving on constrained hardware, serves more decisions per second at lower energy than the filter, while the content of each event remains recoverable from its tokens. We give a rule for choosing between

them and release the pre-registration log, the harness and the analysis, so that the comparison can be repeated for other senders, receivers and tasks.

References

- [1] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, T. Chen, Cut the crap: An economical communication pipeline for LLM-based multi-agent systems, arXiv:2410.02506 (2024).
- [2] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, J. Chen, A review on edge large language models: Design, execution, and applications, ACM Computing Surveys 57 (8) (2025). doi:10.1145/3719664.
- [3] Y. P. Shkolnikov, Agent memory below the prompt: Persistent Q4 KV cache for multi-agent LLM inference on edge devices (2026). arXiv:2603.04428.
- [4] O. Dib, Communication-efficient agentic edge intelligence: A survey of federated multi-agent information fusion for LLM-driven edge AI, Information Fusion 136 (2026) 104577. doi:10.1016/j.inffus.2026.104577.
- [5] J. Dong, W. Lin, BANDMAS: Causality-inspired semantic packet scheduling for bandwidth-efficient multi-agent collaboration, arXiv:2608.00458 (2026).
- [6] J. Mu, X. L. Li, N. Goodman, Learning to compress prompts with gist tokens, in: Advances in Neural Information Processing Systems, 2023.

- [7] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, F. Wei, In-context autoencoder for context compression in a large language model, in: International Conference on Learning Representations, 2024.
- [8] X. Cheng, X. Wang, X. Zhang, T. Ge, S.-Q. Chen, F. Wei, H. Zhang, D. Zhao, xRAG: Extreme context compression for retrieval-augmented generation with one token, in: Advances in Neural Information Processing Systems, 2024.
- [9] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu, LLMlingua: Compressing prompts for accelerated inference of large language models, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2023.
- [10] J. Zou, X. Yang, R. Qiu, G. Li, K. Tieu, P. Lu, K. Shen, H. Tong, Y. Choi, J. He, J. Zou, M. Wang, L. Yang, Latent collaboration in multi-agent systems, accepted, ICML 2026 (Spotlight) (2025). [arXiv: 2511.20639](https://arxiv.org/abs/2511.20639).
- [11] T. Fu, Z. Min, H. Zhang, J. Yan, G. Dai, W. Ouyang, Y. Wang, Cache-to-cache: Direct semantic communication between large language models, in: International Conference on Learning Representations (ICLR 2026), 2025.
URL <https://arxiv.org/abs/2510.03215>
- [12] C. Pham, B. Liu, Y. Yang, Z. Chen, T. Liu, J. Yuan, B. A. Plummer, Z. Wang, H. Yang, Let models speak ciphers: Multiagent debate through embeddings, in: International Conference on Learning Repre-

sentations (ICLR 2024), 2024.

URL https://proceedings.iclr.cc/paper_files/paper/2024/file/e444859b2a22df6b56af9381ad1e9480-Paper-Conference.pdf

- [13] Z. Du, R. Wang, H. Bai, Z. Cao, X. Zhu, Y. Cheng, B. Zheng, W. Chen, H. Ying, Enabling agents to communicate entirely in latent space, in: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL 2026), 2026, pp. 27106–27129. doi:10.18653/v1/2026.acl-long.1248.
- [14] Y. Zheng, Z. Zhao, Z. Li, Y. Xie, M. Gao, L. Zhang, K. Zhang, Thought communication in multiagent collaboration, in: Advances in Neural Information Processing Systems, Vol. 38, 2025, pp. 123389–123418, spotlight.
URL https://proceedings.neurips.cc/paper_files/paper/2025/file/b2b502c3629beadda06311386d2c6f73-Paper-Conference.pdf
- [15] M. Rossi, P. Raghunath, E. Wu, Latent cache flow: Model-to-model communication without text (2026). arXiv:2605.22863.
- [16] C. Flamant, U. Ghai, K. Shimizu, The bicameral model: Bidirectional hidden-state coupling between parallel language models (2026). arXiv:2605.11167.
- [17] H. Jin, P. Kuang, Y. Yu, X. Yuan, H. Wang, Agent primitives: Reusable latent building blocks for multi-agent systems, in: Proceedings of the 43rd International Conference on Machine Learning, Vol. 306 of PMLR,

2026.

URL <https://arxiv.org/abs/2602.03695>

- [18] Y. Liu, Beyond tokens: A unified framework for latent communication in LLM-based multi-agent systems (2026). [arXiv:2606.05711](#).
- [19] L. Dai, L. Xiang, K. Yang, A token/KV-cache communication media selection and resource allocation strategy for multi-agent collaboration, [arXiv:2605.25422](#) (2026).
- [20] X. Liu, R. Zhang, W. Yu, S. Xiong, L. He, F. Wu, H. Jung, M. Fredrikson, X. Wang, J. Gao, The vision wormhole: Latent-space communication in heterogeneous multi-agent systems (2026). [arXiv:2602.15382](#).
- [21] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, Q. Wen, Time-LLM: Time series forecasting by reprogramming large language models, in: International Conference on Learning Representations, 2024.
- [22] N. Babu, J. Mathew, U. Satija, A. P. Vinod, Modality reprogramming: Adapting frozen LLMs for multi-channel EEG classification, *Neurocomputing* 668 (2026) 132407. [doi:10.1016/j.neucom.2025.132407](#).
- [23] Y. Zhao, J. Kang, T. Zhang, P. Han, T. Chen, ECG-Chat: A large ECG-language model for cardiac disease diagnosis, in: 2025 IEEE International Conference on Multimedia and Expo (ICME), 2025, pp. 1–6. [doi:10.1109/ICME59968.2025.11209476](#).
- [24] Y. Zhang, K. Ayush, S. Qiao, A. A. Heydari, G. Narayanswamy, M. A. Xu, A. A. Metwally, S. Xu, J. Garrison, X. Xu, T. Althoff, Y. Liu,

- P. Kohli, J. Zhan, M. Malhotra, S. N. Patel, C. Mascolo, X. Liu, D. McDuff, Y. Yang, SensorLM: Learning the language of wearable sensors, *neurIPS 2025* (2025). [arXiv:2506.09108](#).
- [25] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, A. Y. Ng, Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network, *Nature Medicine* 25 (1) (2019) 65–69.
- [26] K. Singhal, S. Azizi, T. Tu, et al., Large language models encode clinical knowledge, *Nature* 620 (2023) 172–180.
- [27] H. Zhang, M. Emu, Do latent channels actually communicate? a causal audit of latent multi-agent LLM communication, [arXiv:2607.26773](#) (2026).
- [28] R. Lowe, J. Foerster, Y.-L. Boureau, J. Pineau, Y. Dauphin, On the pitfalls of measuring emergent communication, [arXiv preprint arXiv:1903.05168](#) (2019).
- [29] H. Liu, C. Li, Q. Wu, Y. J. Lee, Visual instruction tuning, in: *Advances in Neural Information Processing Systems*, 2023.
- [30] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. L. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Bińkowski, R. Barreira, O. Vinyals, A. Zisserman, K. Simonyan, Flamingo: A visual

- language model for few-shot learning, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 23716–23736.
URL <https://arxiv.org/abs/2204.14198>
- [31] W. Han, T. Chen, C. Duan, X. Song, Y. Yao, Y. Yang, M. A. Rosenberg, E. Liu, D. Zhao, ELF: A family of encoder-free ECG-language models (2026). [arXiv:2601.18798](https://arxiv.org/abs/2601.18798).
 - [32] C. Liu, Z. Wan, C. Ouyang, A. Shah, W. Bai, R. Arcucci, Zero-shot ECG classification with multimodal learning and test-time clinical knowledge enhancement, in: Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of PMLR, 2024, pp. 31949–31963.
URL <https://proceedings.mlr.press/v235/liu24bg.html>
 - [33] S. Wong, R. Shankar, B. Albert, G. Davis Jones, Large language models surpass domain-specific architectures for antepartum electronic fetal monitoring analysis (2025). [arXiv:2509.18112](https://arxiv.org/abs/2509.18112).
 - [34] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Transactions of the Association for Computational Linguistics (2024).
 - [35] C.-Y. Hsieh, Y.-S. Chuang, C.-L. Li, Z. Wang, L. T. Le, A. Kumar, J. Glass, A. Ratner, C.-Y. Lee, R. Krishna, T. Pfister, Found in the middle: Calibrating positional attention bias improves long context utilization, in: Findings of the Association for Computational Linguistics, 2024.

- [36] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, S. Singh, Calibrate before use: Improving few-shot performance of language models, in: Proceedings of the International Conference on Machine Learning, 2021.
- [37] Association for the Advancement of Medical Instrumentation [AAMI], Ansi/aami ec57:1998 – testing and reporting performance results of cardiac rhythm and st segment measurement algorithms (1998).
- [38] G. B. Moody, R. G. Mark, The impact of the MIT-BIH arrhythmia database, IEEE Engineering in Medicine and Biology Magazine 20 (3) (2001) 45–50. doi:10.1109/51.932724.
- [39] P. de Chazal, M. O’Dwyer, R. B. Reilly, Automatic classification of heartbeats using ECG morphology and heartbeat interval features, IEEE Transactions on Biomedical Engineering 51 (7) (2004) 1196–1206. doi:10.1109/TBME.2004.827359.
- [40] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, H. E. Stanley, PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals, Circulation 101 (23) (2000) e215–e220.